任务 1（代码如下）

#导入任务所需要的库

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score
from matplotlib import pyplot as plt
```

#加载任务需要使用到的加载在线的 Iris 数据集

```
url = 'https://www.gairuo.com/file/data/dataset/iris.data'
df = pd.read_csv(url)
```

#查看部分数据集

```
print(df.head())
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

#自行构建特征 (X)和标签 (Y)

```
X = df.iloc[:, :-1].values

Y = df.iloc[:, -1].values
```

#将数据集分成训练集和测试集

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2,
random_state=42)
```

#决策树

```
# 训练决策树模型（使用信息增益 (Information gain) 作为节点的分裂标准）

tree = DecisionTreeClassifier(criterion='entropy', random_state=42）#使用信息增益
(Information gain) 作为节点的分裂标准
```

```
tree.fit(X_train, Y_train)
DecisionTreeClassifier(criterion='entropy', random_state=42)

#报告决策树的深度和叶子节点数，并据此对模型进行可视化

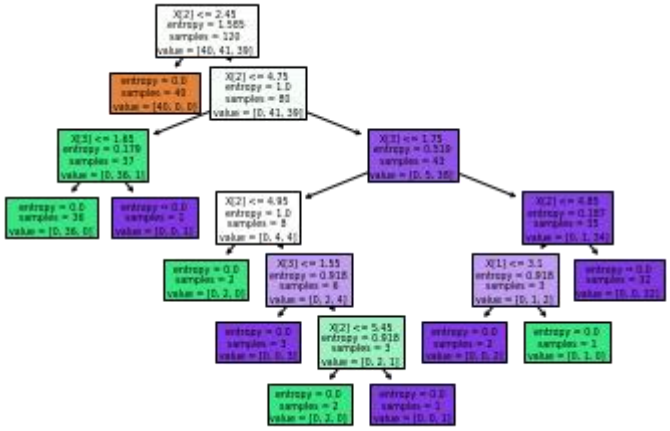
print('决策树的深度: ', tree.tree_.max_depth)
决策树的深度:  6

print('决策树的叶子节点数: ', tree.tree_.n_leaves)
决策树的叶子节点数:  10

plt.figure(figsize=(12, 8))
<Figure size 864x576 with 0 Axes>
<Figure size 864x576 with 0 Axes>

plot_tree(tree, filled=True)
[Text(103.01538461538462, 201.90857142857143, 'X[2] <= 2.45\nentropy = 1.585\nsamples = 120\nvalue = [40, 41, 39]'),
 Text(77.26153846153846, 170.84571428571428, 'entropy = 0.0\nsamples = 40\nvalue = [40, 0, 0]'),
 Text(128.76923076923077, 170.84571428571428, 'X[2] <= 4.75\nentropy = 1.0\nsamples = 80\nvalue = [0, 41, 39]'),
 Text(51.50769230769231, 139.78285714285715, 'X[3] <= 1.65\nentropy = 0.179\nsamples = 37\nvalue = [0, 36, 1]'),
 Text(25.753846153846155, 108.72, 'entropy = 0.0\nsamples = 36\nvalue = [0, 36, 0]'),
 Text(77.26153846153846, 108.72, 'entropy = 0.0\nsamples = 1\nvalue = [0, 0, 1]'),
 Text(206.03076923076924, 139.78285714285715, 'X[3] <= 1.75\nentropy = 0.519\nsamples = 43\nvalue = [0, 5, 38]'),
 Text(128.76923076923077, 108.72, 'X[2] <= 4.95\nentropy = 1.0\nsamples = 8\nvalue = [0, 4, 4]'),
 Text(103.01538461538462, 77.65714285714284, 'entropy = 0.0\nsamples = 2\nvalue = [0, 2, 0]'),
 Text(154.52307692307693, 77.65714285714284, 'X[3] <= 1.55\nentropy = 0.918\nsamples = 6\nvalue = [0, 2, 4]'),
 Text(128.76923076923077, 46.59428571428572, 'entropy = 0.0\nsamples = 3\nvalue = [0, 0, 3]'),
 Text(180.27692307692308, 46.59428571428572, 'X[2] <= 5.45\nentropy = 0.918\nsamples = 3\nvalue = [0, 2, 1]'),
 Text(154.52307692307693, 15.531428571428563, 'entropy = 0.0\nsamples = 2\nvalue = [0, 2, 0]'),
 Text(206.03076923076924, 15.531428571428563, 'entropy = 0.0\nsamples = 1\nvalue = [0, 0, 1]'),
```

```
 Text(283.2923076923077, 108.72, 'X[2] <= 4.85\nentropy = 0.187\nsamples
= 35\nvalue = [0, 1, 34]'),
 Text(257.53846153846155, 77.65714285714284, 'X[1] <= 3.1\nentropy =
0.918\nsamples = 3\nvalue = [0, 1, 2]'),
 Text(231.7846153846154, 46.59428571428572, 'entropy = 0.0\nsamples =
2\nvalue = [0, 0, 2]'),
 Text(283.2923076923077, 46.59428571428572, 'entropy = 0.0\nsamples =
1\nvalue = [0, 1, 0]'),
 Text(309.04615384615386, 77.65714285714284, 'entropy = 0.0\nsamples =
32\nvalue = [0, 0, 32]')]
```



plt.show()

#报告测试集上的准确率（决策树）

Y_pred = tree.predict(X_test)

print('测试集上的准确率（决策树）: ', accuracy_score(Y_test, Y_pred))
测试集上的准确率（决策树）: 1.0

#随机森林

#尝试不同的树的数量（例如，10、50、100），报告相应不同的结果。

n_estimators = [10, 50, 100] # 随机森林的树的数量

训练随机森林模型

```
for n in n_estimators:
    # 训练随机森林模型
    forest = RandomForestClassifier(n_estimators=n, random_state=42)
    forest.fit(X_train, Y_train)
```

```
    # 报告测试集上的准确率（随机森林）
    Y_pred = forest.predict(X_test)
    print(f'树的数量（随机森林）: {n}, 测试集上的准确率（随机森林）:',
accuracy_score(Y_test, Y_pred))

    # 报告特征重要性（随机森林）
    print('特征重要性（随机森林）:', forest.feature_importances_)
```

```
树的数量（随机森林）: 10, 测试集上的准确率（随机森林）:  1.0
特征重要性（随机森林）:  [0.14908549 0.02575563 0.42600688 0.39915201]
树的数量（随机森林）: 50, 测试集上的准确率（随机森林）:  1.0
特征重要性（随机森林）:  [0.11159383 0.02860807 0.44578786 0.41401024]
树的数量（随机森林）: 100, 测试集上的准确率（随机森林）:  1.0
特征重要性（随机森林）:  [0.10809762 0.03038681 0.43999397 0.42152159]
```

```
#梯度提升树

# 梯度提升树的学习率和树的数量

learning_rates = [0.01, 0.1, 1]

n_estimators = [50, 100, 150]

for lr in learning_rates:
    for n in n_estimators:
        # 训练梯度提升树模型
        gbt = GradientBoostingClassifier(learning_rate=lr, n_estimators=n,
random_state=42)
        gbt.fit(X_train, Y_train)

        # 报告测试集上的准确率（梯度提升树）
        Y_pred = gbt.predict(X_test)
        print(f'学习率（梯度提升树）: {lr}, 树的数量（梯度提升树）: {n}, 测
试集上的准确率（梯度提升树）:', accuracy_score(Y_test, Y_pred))

        # 报告特征重要性（梯度提升树）
        print('特征重要性（梯度提升树）:', gbt.feature_importances_)
```

```
学习率（梯度提升树）: 0.01, 树的数量（梯度提升树）: 50, 测试集上的准确
率（梯度提升树）:  1.0
特征重要性(梯度提升树):  [2.73281525e-06 4.16229554e-07 6.48309120e-01
3.51687731e-01]
学习率（梯度提升树）: 0.01, 树的数量（梯度提升树）: 100, 测试集上的准
确率（梯度提升树）:  1.0
```

特征重要性(梯度提升树): [5.85338898e-04 4.15011638e-03 6.27971002e-01 3.67293542e-01]
学习率（梯度提升树）: 0.01, 树的数量（梯度提升树）: 150, 测试集上的准确率（梯度提升树）: 1.0
特征重要性(梯度提升树): [0.00071947 0.0096489 0.61643974 0.37319189]
学习率（梯度提升树）: 0.1, 树的数量（梯度提升树）: 50, 测试集上的准确率（梯度提升树）: 1.0
特征重要性(梯度提升树): [0.00128451 0.01461426 0.67166104 0.3124402]
学习率（梯度提升树）: 0.1, 树的数量（梯度提升树）: 100, 测试集上的准确率（梯度提升树）: 1.0
特征重要性(梯度提升树): [0.00134833 0.0146823 0.67151452 0.31245486]
学习率（梯度提升树）: 0.1, 树的数量（梯度提升树）: 150, 测试集上的准确率（梯度提升树）: 1.0
特征重要性(梯度提升树): [0.00134841 0.01468262 0.67151409 0.31245488]
学习率（梯度提升树）: 1, 树的数量（梯度提升树）: 50, 测试集上的准确率（梯度提升树）: 1.0
特征重要性（梯度提升树）: [0.0025056 0.00421749 0.9041221 0.08915481]
学习率（梯度提升树）: 1, 树的数量（梯度提升树）: 100, 测试集上的准确率（梯度提升树）: 1.0
特征重要性（梯度提升树）: [0.0025056 0.00421749 0.9041221 0.08915481]
学习率（梯度提升树）: 1, 树的数量（梯度提升树）: 150, 测试集上的准确率（梯度提升树）: 1.0
特征重要性（梯度提升树）: [0.0025056 0.00421749 0.9041221 0.08915481]

任务 2

1. 决策树、随机森林和梯度提升树在测试集上的准确率均达到了 1.0，无法判断哪个最好。但一般来说随机森林和梯度提升树的性能要优于单一决策树，他们能采用集成方法来降低过拟合的情况并提升预测性能，并且随机森林通过增加树的数量来投票或加权平均进行预测，增强了模型的稳定性和准确性。

2. 在随机森林和梯度提升树中，第三和第四个特征对预测最为重要。这可能是因为这两个特征与目标变量关联度高，或者它们能更有效地区分不同类别。

3. 特征的重要性通常是通过计算模型中所有树的平均不纯度减少来确定的。如果一个特征在许多树中用于分裂，那么这个特征被认为是重要的。因为如果一个特征能够提供有用的信息来预测目标变量，模型就会更频繁地使用它进行分裂。

4. 梯度提升树对超参数选择最为敏感。决策树的主要超参数是树的深度，随机森林的主要超参数是树的数量和特征的数量。然而，梯度提升树除了这些超参数外，还有学习率和损失函数等超参数。因此，梯度提升树的性能更容易受到超参数选择的影响。